

UniConnect — Team Presentation Guide

Team Mavericks · UIU · Dhaka · 2026–27

Members: Joydip Datta (Leader) · Md. Saem Ferdous · Md. Mahfujur Rahman Himel Akon · Md. Monabbur Hosen Bhuiyan · Md. Nazmul Hasan Nasim

This document maps every feature to a team contributor, explains how each system works end-to-end, and prepares the team for every question a teacher might ask during the presentation.

Table of Contents

- 1. Project Overview
- 2. System Architecture
- 3. Tech Stack
- 4. Five Contribution Areas
- 5. Feature Deep-Dives (How Everything Works)
- 6. Database Design
- 7. Teacher Q&A Bank

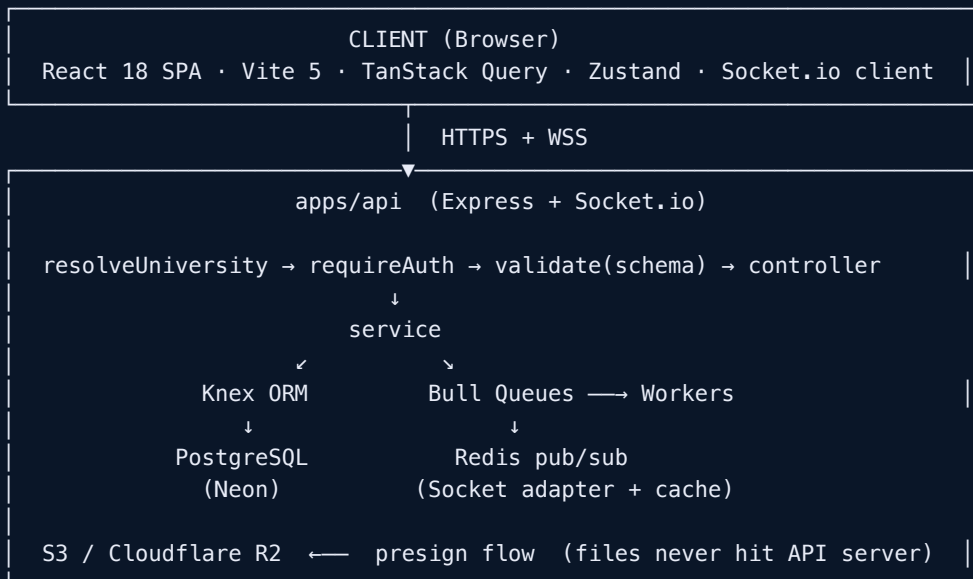
1. Project Overview

UniConnect is a private, multi-tenant university social network — think LinkedIn but locked to a single university's students, alumni, faculty, and admins. Each university gets its own isolated instance (tenant) sharing the same codebase and database cluster.

Metric	Value
Backend modules	21
Database migrations	81
Frontend pages	30+
React feature bundles	22
Background job queues	10
User roles	student, alumni, faculty, admin, driver
Lines of backend TypeScript	~15,000
Lines of frontend TypeScript/TSX	~18,000

Target users: Students (social feed, events, jobs), Alumni (mentorship, job posting), Faculty/Admin (content moderation, group management), Transport Staff/Driver (GPS shuttle broadcast).

2. System Architecture



Multi-tenancy: every DB table has `university_id` UUID FK.
No Postgres RLS – isolation enforced in service layer.
University resolved from `x-university-domain` HTTP header.

Monorepo:

`apps/web` React SPA
`apps/api` Express backend
`packages/shared` shared Zod schemas, TypeScript types, socket event constants

Data flow for a typical POST `/feed/posts`:

Browser

- `axios` (injects `x-university-domain` + `Authorization: Bearer <token>`)
- Express router: `resolveUniversity` → `requireAuth` → `validate(createPostSchema)`
- `feedController.createPost`
- `feedService.createPost` (DB write via Knex)
- `notificationQueue.add(...)` (Bull enqueue – async, non-blocking)
- `getIo().to('uni:'+universityId).emit('NEW_POST', post)` (real-time)
- `sendSuccess(res, post, 201)`

3. Tech Stack

Layer	Technology	Why chosen
Frontend framework	React 18 + Vite 5	Fast HMR, lazy-loading via React.lazy
Styling	Tailwind CSS + CSS custom properties	Utility classes + design tokens without runtime overhead
State (server)	TanStack Query v5	Cache, background refetch, optimistic updates
State (client)	Zustand	Minimal boilerplate, no context drilling
Real-time	Socket.io	Rooms, auto-reconnect, fallback transports
Backend	Express + Node.js (TypeScript strict)	Mature ecosystem, team familiarity
Database	PostgreSQL (Neon cloud)	ACID, full-text search (tsvector/GIN), JSON columns
Query builder	Knex	Migrations + query building without ORM overhead
Queue/Cache	Redis + Bull	Reliable job processing, pub/sub for Socket.io scale
File storage	AWS S3 / Cloudflare R2	Presigned URL pattern; files never pass through API
Email	Resend API	Transactional email for OTPs and notifications
Deployment	Azure App Service (API) + Vercel (Web)	GitHub Actions CI/CD

4. Five Contribution Areas

Rule: Leader owns the most cross-cutting, architecturally critical work. Others own well-scoped feature verticals. Every member owns both backend (module) and frontend (feature bundle) for their area.

Team Roster

#	Name	Area	Headline responsibilities
1	Joydip Datta (Leader)	Core Architecture, Auth, Real-time & Admin	Monorepo, multi-tenancy, JWT/OTP auth, Socket.io+Redis, all Bull queues, admin panel, file upload, rate limiting, moderation, drafts, account deletion, design system, CI/CD
2	Md. Saem Ferdous	Social Feed, Groups & Notifications	Feed + hot_score ranking, post lifecycle, groups, notifications, news, badges, share
3	Md. Mahfujur Rahman Himel Akon	Jobs, Events, Connections, Profile & Onboarding	Jobs, events, connection graph, full profile suite, onboarding checklist, mentorship + points economy
4	Md. Monabbur Hosen Bhuiyan	Messaging, Presence & Push	Real-time chat, conversation types, Redis presence, privacy tiers, Web Push
5	Md. Nazmul Hasan Nasim	Campus Tools, Search & Content	Full-text search, explore, campus (lost-found + shuttle GPS), content-sync (WordPress + Skyvern), settings

Contributor 1 — Joydip Datta (LEADER): Core Architecture, Auth, Real-time & Admin

This is the hardest area. It underpins everything else. Nothing in the project works without this.

Owned systems:

System	Files / Location
Monorepo setup	<code>pnpm-workspace.yaml</code> , <code>package.json</code> root
Multi-tenancy middleware	<code>apps/api/src/middleware/university.ts</code>
JWT Auth (access + refresh)	<code>apps/api/src/modules/auth/</code> · <code>apps/api/src/services/token.service.ts</code>
OTP verification	<code>apps/api/src/services/otp.service.ts</code>
Invite-based registration	<code>apps/api/src/modules/auth/service.ts</code>
Socket.io + Redis pub/sub	<code>apps/api/src/socket/index.ts</code>
All 10 Bull queues + workers	<code>apps/api/src/queues/</code> · <code>apps/api/src/workers/</code>
Admin module (full)	<code>apps/api/src/modules/admin/</code>
Database schema (001–029)	<code>apps/api/src/database/migrations/001–029_*.ts</code>
Error handling middleware	<code>apps/api/src/middleware/error-handler.ts</code>
Response helpers	<code>apps/api/src/utils/response.ts</code> · <code>errors.ts</code>
Winston logger	<code>apps/api/src/utils/logger.ts</code>
Design system (tokens)	<code>apps/web/src/styles/tokens.css</code>
Theme switcher	<code>apps/web/src/stores/themeStore.ts</code>
CI/CD	<code>.github/workflows/</code>
File upload (presigned S3)	<code>apps/api/src/modules/upload/</code> · <code>apps/api/src/services/upload.service.ts</code>
Rate limiting (Redis, per-user)	<code>apps/api/src/middleware/rateLimiter.ts</code>
Moderation (block / mute / report)	<code>apps/api/src/modules/moderation/</code> · migration 078
Account deletion requests	<code>apps/api/src/modules/users/</code> (deletion sub-API) · migration 079
Drafts (unified cross-domain)	<code>apps/api/src/modules/drafts/</code> · migration 064 · <code>apps/web/src/features/drafts/</code>
Auth frontend	<code>apps/web/src/features/landing/</code> · <code>LoginPage.tsx</code> · <code>RegisterPage.tsx</code>
Admin frontend	<code>apps/web/src/pages/admin/</code> · <code>apps/web/src/features/</code>

How file upload works (leader must know this cold):

Presigned S3 PUT URL flow – file bytes never hit the API server:

1. Client: POST /api/upload/presign { filename, contentType }
2. API: upload.service.ts → S3.createPresignedPost() → returns { url, fields } (signed, 5 min)
3. Client: PUT directly to S3/R2 URL (multipart form, bypasses Express entirely)
4. Client: sends resulting public URL to domain endpoint
e.g. PATCH /users/me { avatar_url: 'https://cdn.../abc.jpg' }
5. API: stores URL string in DB – no file bytes ever enter Node.js process

Why this matters: a 10 MB video upload doesn't block the Express event loop or consume Lambda/App Service memory. The API just signs a token and walks away.

How auth works (leader must know this cold):

Registration:

- ```
POST /auth/register (body: token, email, password, name)
 → validate invite token exists + email matches
 → bcrypt.hash(password)
 → insert user + profile in transaction
 → otpService.generate(userId, 'email_verify') → store in Redis (TTL 10 min)
 → emailQueue.add({ to, otp }) → email.worker sends via Resend API
 → return { message: 'OTP sent' }
```

POST /auth/verify-otp

- ```
  → otpService.verify(userId, otp, purpose)
  → set users.is_verified = true
  → return { accessToken, user } + Set-Cookie: refreshToken (httpOnly, 7d)
```

Login:

- ```
POST /auth/login
 → bcrypt.compare(password, hash)
 → tokenService.signAccess({ userId, universityId, role }) (JWT 15 min)
 → tokenService.signRefresh() → 256-bit random → stored in user_sessions
 → Set-Cookie: refreshToken (httpOnly)
```

Silent refresh (frontend interceptor):

- ```
  axios 401 interceptor → check localStorage 'uc:has_session' sentinel
  → POST /auth/refresh (cookie sent automatically)
  → new accessToken → retry original request
```

How moderation & rate limiting work (leader-owned safety layer):

Moderation module (user-level trust & safety, not just admin):

- ```
POST /moderation/block/:userId - hide a user both ways (no posts, no DMs)
DELETE /moderation/block/:userId - unblock
POST /moderation/mute/:userId - silence a user (you stop seeing them; they don't know)
DELETE /moderation/mute/:userId - unmute
POST /moderation/report - file a content/user report → lands in admin queue
GET /moderation/blocks · /mutes - manage your lists
 → backed by user_moderation table (migration 078)
 → admin resolves reports in the admin panel (university_audit_log records the action)
```

Rate limiting (rateLimiter.ts – Redis sliding counter):

- ```
key = rl:{prefix}:{subject} where subject = u:{userId} (authenticated) or ip:{ip} (anon)
  → INCR key with a TTL window; reject with 429 (tooManyRequests) over the limit
  → keyed by USER id, not IP, so a whole university behind one NAT isn't throttled as one client
  → writeLimiter guards all mutating routes (block, mute, report, post, etc.)
  → disabled in NODE_ENV=test (noop) so tests aren't throttled
```

Contributor 2 — Md. Saem Ferdous: Social Feed, Groups & Notifications

Owned systems:

System	Files / Location
Feed (posts, reactions, comments, polls)	<code>apps/api/src/modules/feed/</code>
Feed ranking (hot_score)	<code>apps/api/src/workers/feed-ranking.worker.ts</code> · migration <code>071</code>
Post lifecycle (draft→scheduled→published→archived)	<code>apps/api/src/workers/post-lifecycle.worker.ts</code> · migration <code>072</code>
Groups (join-request, roles, resources, study sessions)	<code>apps/api/src/modules/groups/</code>
Group digest (weekly Bull cron)	<code>apps/api/src/workers/group-digest.worker.ts</code>
Notifications	<code>apps/api/src/modules/notifications/</code>
News module	<code>apps/api/src/modules/news/</code>
Tags / hashtags	migration <code>013</code> · <code>preprocessHashtags.ts</code>
Share feature	<code>apps/web/src/features/share/</code>
Badge system	<code>apps/api/src/workers/badge.worker.ts</code>
Feed frontend	<code>apps/web/src/features/feed/</code>
Groups frontend	<code>apps/web/src/features/groups/</code>
Notifications frontend	<code>apps/web/src/features/notifications/</code>
News frontend	<code>apps/web/src/features/news/</code>

How feed ranking works:

Columns on posts table (migration 071):

- `reaction_count` – incremented transactionally on every reaction write
- `comment_count` – incremented transactionally on every comment write
- `hot_score` – precomputed Hacker News-style score

Hot score formula (`feed-ranking.worker.ts`):

$$\text{score} = (\text{reactions} + \text{comments} * 2) / ((\text{age_hours} + 2) ^ 1.8)$$

- Bull cron runs every hour → recalculates `hot_score` for all posts
- "Top" feed sorts on `idx_posts_hot_score` index (no subquery cost)
- "Recent" feed sorts on `created_at` DESC

Post lifecycle (`post-lifecycle.worker.ts`):

- Bull cron every minute scans posts
- `publish_at` in past + `is_published=false` → flip to published + emit SOCKET event
- `archived_at` in past + `is_published=true` → flip to archived
- States: draft (no `publish_at`, `is_published=false`)
 - scheduled (`publish_at` in future)
 - published (`is_published=true`)
 - archived (`archived_at` in past)

How groups work:

```
Group types: public, private, restricted + system_group flag
Private join flow:
  POST /groups/:id/join →
    if public: insert member immediately
    if private: insert group_join_requests (status='pending')
                  → notify group admins
  PATCH /groups/:id/join-requests/:reqId (admin only):
    status='approved' → insert into group_members
    status='rejected' → update join request

Member roles: admin > moderator > member
Resources: file links stored in group_resources (view tracking via group_resource_views)
Study sessions: group_study_sessions table + RSVP via group_session_rsmps
```

Contributor 3 — Md. Mahfujur Rahman Himel Akon: Jobs, Events, Connections, Profile & Onboarding

Owned systems:

System	Files / Location
Jobs (listings, applications, saved)	<code>apps/api/src/modules/jobs/</code>
Events (create, RSVP, group-linked)	<code>apps/api/src/modules/events/</code>
Connections (bidirectional graph)	<code>apps/api/src/modules/connections/</code> · migration <code>051</code>
Profile (extended sections)	<code>apps/api/src/modules/users/</code> (profile sub-API)
Experience entries	table <code>profile_experiences</code> · migration <code>053</code>
Education entries	table <code>profile_education</code> · migration <code>054</code>
Featured items	table <code>profile_featured</code> · migration <code>055</code>
Profile analytics & viewers	table <code>profile_views</code> · migration <code>057</code>
Resume export	<code>apps/web/src/features/profile/ResumeExportButton.tsx</code>
Profile vanity URLs	<code>users.username</code> unique per university · migration <code>072</code>
Onboarding checklist & profile progress	<code>GET /users/me/progress</code> · <code>profileProgressSchema</code> · <code>apps/web/src/features/onboarding/</code>
Mentorship module	<code>apps/api/src/modules/mentorship/</code>
Mentorship points economy	<code>POINTS_PER_SESSION=10</code> · <code>POINTS_PER_USD=100</code>
Gift card redemption	<code>mentorship_redemptions</code> table
Jobs frontend	<code>apps/web/src/features/jobs/</code>
Events frontend	<code>apps/web/src/features/events/</code>
Connections frontend	<code>apps/web/src/features/connections/</code>
Profile frontend	<code>apps/web/src/features/profile/</code>
Mentorship frontend	<code>apps/web/src/features/mentorship/</code>

How connections work:

Old model: follows table (one-directional, dropped in migration 052)

New model: connections table (bidirectional LinkedIn-style)

```
connections(id, university_id, requester_id, recipient_id, status, note)
status: 'pending' | 'accepted'
```

Flow:

POST /connections/request/:userId

- insert connection(requester=me, recipient=target, status='pending')
- emit CONNECTION_EVENTS.REQUEST_RECEIVED to user:{targetId} room

POST /connections/:connectionId/accept

- update status='accepted'
- emit CONNECTION_EVENTS.ACCEPTED to user:{requesterId} room

GET /connections/mutual/:userId

- find users connected to BOTH me AND target userId
- two-hop intersection query

Connection status on profiles:

getUser() checks connections table → returns 'none' | 'pending_sent' | 'pending_received' | 'connected' so the ConnectButton renders correctly

How mentorship points economy works:

Every completed mentorship session → +10 points to mentor (alumni)

Points can be redeemed for gift cards:

gift_cards table: vendor, title, threshold_points, value_usd_cents

POST /mentorship/redeem

- check user.mentorship_points >= gift_card.threshold_points
- deduct points → insert mentorship_redemptions
- admin fulfils by adding code_text + marking fulfilled

Capacity: alumni sets max_mentees on profile (default 3)

Accept request → count active mentees → reject if at capacity

How onboarding / profile progress works:

GET /users/me/progress → profileProgressSchema (packages/shared)

Returns first-run signals that drive the onboarding checklist:

profileScore	0-100 weighted completion of profile fields
isVerified	email OTP completed
hasAvatar / hasBio / hasHeadline	granular per-field flags
hasAddedExperience / hasAddedEducation	
hasMadePost	has published at least one post
connectionCount	number of accepted connections

Frontend: OnboardingChecklist.tsx reads these flags and deep-links the user straight to the exact gap (e.g. "Add a bio" → opens EditProfileModal).

The checklist disappears once profileScore hits 100.

Contributor 4 — Md. Monabbur Hosen Bhuiyan: Messaging, Real-time Presence & Push Notifications

Owned systems:

System	Files / Location
Messaging (DMs, group chat, mentorship threads)	<code>apps/api/src/modules/messages/</code>
Conversation types (direct, group, mentorship)	<code>conversations.type</code> discriminator
Presence (Redis-based online status)	<code>apps/api/src/modules/presence/</code>
Privacy tiers for visibility	<code>apps/web/src/features/settings/PrivacySection.tsx</code>
Web Push (VAPID subscriptions)	<code>apps/api/src/modules/push/</code> · migration <code>066</code>
Push worker (fan-out)	<code>apps/api/src/workers/push.worker.ts</code>
Notification digest (Bull cron)	<code>apps/api/src/workers/notification-digest.worker.ts</code>
Pending messages store (offline queue)	<code>apps/web/src/stores/pendingMsgsStore.ts</code>
Socket store	<code>apps/web/src/stores/socketStore.ts</code>
Presence store	<code>apps/web/src/stores/presenceStore.ts</code>
Messages frontend	<code>apps/web/src/features/messages/</code>
Presence frontend	<code>apps/web/src/features/presence/</code>

How real-time messaging works:

```
Socket.io rooms:
uni:{universityId}    - broadcast to whole university (feed posts, events)
user:{userId}         - personal (notifications, connection requests)
conv:{conversationId} - chat messages

Message flow:
Client sends: socket.emit('SEND_MESSAGE', { convId, content })
Server:
  → verify user is participant in conversation
  → INSERT into messages table
  → io.to('conv:'+convId).emit('NEW_MESSAGE', message)
  → notificationQueue.add() for offline participants

Offline messages:
pendingMsgsStore buffers outgoing messages when socket disconnected
→ on reconnect, flushes buffer → server deduplicates by client_id

Mentorship conversations:
When mentorship request accepted:
  → INSERT into conversations(type='mentorship', mentorship_request_id=X)
  → Both parties auto-joined to conversation
  → mentorship_requests.conversation_id FK set
```

How presence works:

```
Redis keys:
  presence:count:{userId}      - socket count (int, TTL refreshed by heartbeat)
  presence:online:{universityId} - SET of online user IDs
```

```
Connect (0→1 transition):
  socket.on('connection') → registerConnect(userId, universityId)
  → INCR presence:count:{userId}
  → if result === 1: SADD presence:online:{universityId}, userId
                    → broadcastPresence(userId, 'online')
                    emit PRESENCE_EVENTS.ONLINE to uni room
```

```
Heartbeat:
  Client emits PRESENCE_EVENTS.PING every 30s
  → server: EXPIRE presence:count:{userId} 120 (refresh TTL)
```

```
Disconnect (1→0 transition):
  registerDisconnect → DECR count
  → if result === 0: SREM online set
                    UPDATE users.last_seen = now()
                    emit PRESENCE_EVENTS.OFFLINE
```

```
Privacy tiers (OnlineVisibilityTier):
  everyone | connections_only | nobody
  → presence.service enforces tier when returning GET /presence
```

Contributor 5 — Md. Nazmul Hasan Nasim: Campus Tools, Search, Content-Sync & Settings

Owned systems:

System	Files / Location
Full-text search (GIN indexes)	<code>apps/api/src/modules/search/</code> · migration <code>070</code>
pg_trgm fuzzy search fallback	migration <code>024</code>
Explore module (trending, discover)	<code>apps/api/src/modules/explore/</code>
Campus module (lost-and-found + shuttle)	<code>apps/api/src/modules/campus/</code>
Shuttle GPS broadcast (driver role)	<code>POST /shuttle/locations</code> · <code>apps/web/src/pages/ShuttleDrivePage.tsx</code>
Content-sync (WordPress + Skyvern)	<code>apps/api/src/modules/content-sync/</code>
Content-sync worker	<code>apps/api/src/workers/content-sync.worker.ts</code>
Skyvern browser-automation fallback	<code>apps/api/src/services/skyvern.service.ts</code>
Settings (notification + privacy JSONB)	<code>apps/api/src/modules/users/</code> (settings sub-API) · migration <code>065</code>
Search frontend	<code>apps/web/src/features/search/</code>
Campus / lost-found / shuttle frontend	<code>apps/web/src/features/lost-found/</code> · <code>shuttle/</code>
Settings frontend	<code>apps/web/src/features/settings/</code>
Content-sync frontend (admin)	<code>apps/web/src/features/content-sync/</code>

How full-text search works:

Migration 070 adds STORED generated columns on each searchable table:

```
posts.search_vector TSVECTOR GENERATED ALWAYS AS (
  setweight(to_tsvector('english', content), 'C')
) STORED
```

```
profiles.search_vector TSVECTOR GENERATED ALWAYS AS (
  setweight(to_tsvector('simple', full_name), 'A') ||
  setweight(to_tsvector('english', COALESCE(headline, '')), 'B') ||
  setweight(to_tsvector('english', COALESCE(bio, '')), 'C')
) STORED
```

GIN index on each search_vector column → O(1) lookup instead of O(n) scan

Search query:

```
GET /api/v1/search?q=machine+learning&type=posts,users
→ service builds: WHERE search_vector @@ plainto_tsquery('english', ?)
→ ORDER BY ts_rank(search_vector, query) DESC
→ fallback: WHERE full_name ILIKE '%?%' (pg_trgm)
```

Multi-entity search returns typed results:

```
{ users: [...], posts: [...], jobs: [...], events: [...], groups: [...] }
```

How content-sync works:

```
Admin configures WordPress REST API URLs per category (news/notices/events)
PATCH /api/v1/admin/content-sync/config → university_settings updated
```

```
POST /api/v1/admin/content-sync/run
→ contentSyncQueue.add({ universityId })
→ content-sync.worker picks up job:
  1. Fetch WordPress REST API: GET {url}?per_page={entriesPerSource}
  2. Parse JSON → extract title, content, date, featured_image
  3. If WP fetch fails → skyvern.service.ts fallback:
      Skyvern browser automation: navigate URL, extract DOM, return structured data
  4. Deduplicate by external_id (idempotent)
  5. INSERT into news/events table with is_published=false
  6. Emit CONTENT_SYNC_EVENTS.COMPLETED via socket
  7. Log run in content_sync_runs table
```

```
Admin reviews imported items at GET /api/v1/admin/content-sync/pending
→ publishes selected items → visible to students
```

5. Feature Deep-Dives

5.1 Multi-tenancy

The entire platform serves multiple universities from a single codebase and database.

```
Every domain table: university_id UUID FK + indexed
Request lifecycle:
  1. Client sends x-university-domain: uiu.ac.bd header
  2. resolveUniversity middleware: SELECT * FROM universities WHERE domain = ?
  3. Sets req.university = { id, name, domain }
  4. All service calls receive universityId from req.university.id
      NEVER from req.body (security: prevents tenant hopping)

University data is fully isolated at query level:
db('posts').where({ university_id: universityId, ... })
```

5.2 File Upload (Presign Flow)

File bytes never travel through the API server — critical for performance.

```
1. Client: POST /api/upload/presign { filename, contentType }
2. API: S3.createPresignedPost() → returns { url, fields } (signed, expires 5 min)
3. Client: PUT directly to S3/R2 URL (multipart form, bypasses API)
4. Client: sends the resulting public URL to the domain endpoint
   e.g. PATCH /users/me { avatar_url: 'https://cdn.../abc.jpg' }
5. API: stores URL in DB — no file bytes ever hit Node.js process
```

5.3 Background Jobs (Bull + Redis)

10 queues, each with a dedicated worker process (or in-process in production):

Queue	Trigger	What worker does
email	OTP, welcome, password reset	Sends via Resend API
notification	Post reactions, comments, mentions	Creates DB notification rows
notification-digest	Daily cron	Emails unread notification summaries
badge	Points earned, milestones hit	Awards badges, updates leaderboard
group-digest	Weekly cron	Sends group activity summary email
mentorship	Request accepted or 48h idle	Reminder emails + 7-day auto-expiry
push	Notification created	Web Push fan-out to all user subscriptions
content-sync	Admin triggers	WordPress fetch → Skyvern fallback
feed-ranking	Hourly cron	Recomputes hot_score on all posts
post-lifecycle	Every minute cron	Flips scheduled→published, published→archived

5.4 Design System

Single source of truth: `apps/web/src/styles/tokens.css`

Two themes: Warm Futuristic Dark (default) + Warm Neutral Light (`data-theme='light'`)

Brand colors:

```
--uc-orange: #F05A28  (UIU identity)
--uc-indigo:  #5B5BD6  (interactive elements)
--uc-navy:   #1E3A70  (accent surfaces)
```

Surface stacking (no box-shadow, depth via background):

```
--surface-page   #060D1A  (outermost)
--surface-card   #0A1628  (cards)
--surface-raised #111D35  (modals, dropdowns)
```

Rules enforced in every component:

- No hardcoded hex – always `var(--token-name)`
- Border: 0.5px solid `var(--border-*)` – never 1px structural
- Buttons: `border-radius: var(--r-pill)` – no sharp corners
- Font weight: 400 and 500 only – never 600+
- Sentence case everywhere – no ALL CAPS

5.5 Driver Role & Shuttle System

A minimal-privilege role added specifically for transport staff.

Migration 073:

users.role enum gains 'driver'

shuttle_routes gains: est_duration_min, cycle_minutes (client-side interpolation params)

Driver flow:

ShuttleDrivePage.tsx → navigator.geolocation.watchPosition()

→ POST /shuttle/locations every 5s { route_id, lat, lng }

→ only 'driver' role can call this endpoint (requireRole('driver'))

→ socket emits SHUTTLE_LOCATION to uni:{universityId} room

→ ShuttlePage.tsx receives live coords, interpolates bus position on route

using est_duration_min + cycle_minutes without needing a real GPS server

6. Database Design

81 migrations building a schema from scratch. Key design decisions:

UUID primary keys: id UUID DEFAULT uuid_generate_v4()

Timestamps: created_at / updated_at TIMESTAMPTZ DEFAULT now()

Soft deletes: is_deleted BOOLEAN DEFAULT false (never deleted_at)

Indexes: every FK column + (university_id, created_at DESC) on high-volume tables

JSONB columns: user_settings.notification_preferences, user_settings.privacy_preferences

Generated columns: search_vector TSVECTOR GENERATED ALWAYS AS (...) STORED (migration 070)

No Postgres RLS: all isolation via service-layer WHERE university_id = ?

Key schema groups:

Group	Tables
Core	universities, university_settings, users, profiles, invitations, user_sessions
Feed	posts, comments, reactions, saved_posts, tags, post_tags, polls, poll_votes
Jobs	jobs, job_applications
Events	events, event_rsmps
Groups	groups, group_members, group_join_requests, group_resources, group_study_sessions, group_session_rsmps
Messaging	conversations, conversation_participants, messages, message_reads
Notifications	notifications, news
Mentorship	mentorship_requests, mentorship_sessions, mentorship_feedback, gift_cards, mentorship_redemptions
Connections	connections
Profile	profile_experiences, profile_education, profile_featured, profile_views
Campus	lost_found_items, shuttle_routes, shuttle_schedules, shuttle_locations
Push	push_subscriptions
Settings	user_settings
Sync	content_sync_runs, content_attachments
Audit	university_audit_log
Moderation	user_moderation, account_deletion_requests

7. Teacher Q&A Bank

Architecture & Design

Q: Why did you choose a monorepo structure?

A: We have three packages — `apps/web` , `apps/api` , and `packages/shared` . The shared package contains Zod schemas and TypeScript types that both the frontend and backend use. Without a monorepo, we'd have to publish the shared package to npm or duplicate types. With pnpm workspaces and a monorepo, a single `import { createPostSchema } from '@uniconnect/shared'` works in both apps. This eliminates schema drift between client and server validation.

Q: How does multi-tenancy work? Can University A see University B's data?

A: No. Every table has a `university_id` UUID foreign key. The `resolveUniversity` middleware runs on every request, resolves the university from the `x-university-domain` header, and sets `req.university.id` . Every service call then filters by that ID — `db('posts').where({`

`university_id: universityId })` . We never read `university_id` from the request body. There is no Postgres Row-Level Security; we chose application-level isolation for simpler debugging.

Q: Why not use an ORM like Prisma or TypeORM?

A: We use Knex, which is a query builder, not an ORM. Knex gives us direct control over SQL, handles migrations cleanly with a numbered file system, and doesn't add the weight of ORM entity mapping. Since we have complex multi-tenant queries with dynamic `university_id` filters and full-text search using Postgres-specific features like `tsvector` , raw SQL proximity is valuable. We drop to raw SQL only when Knex genuinely can't express the query.

Q: How do you ensure the frontend never calls the wrong university's API?

A: The Vite frontend has an environment variable `VITE_UNIVERSITY_DOMAIN=uiu.ac.bd` . The Axios instance (in `apps/web/src/lib/axios.ts`) automatically injects `x-university-domain: uiu.ac.bd` on every HTTP request. On the server, the middleware reads this header and resolves the university — never from any user-controlled body parameter.

Authentication & Security

Q: How are JWT tokens stored? Isn't localStorage insecure?

A: The access token (15 min lifetime) is stored in Zustand `authStore` — memory only. It lives in JavaScript heap, not localStorage, so it's not accessible to XSS scripts reading `localStorage` . The refresh token (7 days) is an httpOnly cookie, which JavaScript cannot read at all. `localStorage` stores only a sentinel flag `uc:has_session` — a boolean, not a token — so the axios interceptor knows whether to attempt a silent refresh on 401.

Q: What happens when the access token expires?

A: The axios interceptor catches every 401 response. If the session sentinel exists in localStorage, it calls `POST /auth/refresh` — the browser automatically sends the httpOnly refresh cookie. The server validates the refresh token against `user_sessions` table (prevents reuse), issues a new access token, and the interceptor retries the original request. The user never notices. If the refresh also fails (e.g. token revoked), the user is logged out.

Q: Why do you use an invite token for registration?

A: UniConnecT is a private network. Students can't self-register — they need an invite from an admin. This prevents random external users from joining. The invite contains a pre-set email and role, so a student can't register with an alumni invitation. Admins can bulk-create invitations via the admin panel.

Q: How does OTP work?

A: After register, the server generates a 6-digit OTP, stores it in Redis with a 10-minute TTL keyed to the `userId + purpose` (e.g. `otp:email_verify:{userId}`), and sends it via email (Resend API → email queue → email worker). `POST /auth/verify-otp` checks the Redis key, marks the user verified, and deletes the key. Redis ensures OTPs expire automatically without a cleanup cron.

Q: How are passwords stored?

A: Using bcrypt with a cost factor of 12. We never store plaintext passwords. The JWT payload contains only `{ userId, universityId, role }` — no sensitive data. The `JWT_SECRET` and `JWT_REFRESH_SECRET` are separate secrets, so a refresh token cannot be used as an access token.

Q: How do you protect the API from abuse / spam?

A: A Redis-backed rate limiter (`rateLimiter.ts`) guards every mutating route. It uses a sliding counter keyed by `rl:{prefix}:{subject}` where the subject is the authenticated user id (`u:{userId}`) when available, falling back to the IP for anonymous routes like login. Keying by user id is deliberate — an entire university sitting behind one NAT IP would otherwise be throttled as a single client. Over the limit returns HTTP 429. It's a no-op in the test environment so tests aren't throttled.

Q: How can users protect themselves from harassment?

A: The moderation module gives users two tools plus reporting. **Block** (`POST /moderation/block/:userId`) hides the user both ways — no posts, no DMs in either direction. **Mute** silences a user one-directionally — you stop seeing their content but they aren't notified. **Report** files a content or user report that lands in the admin review queue. All three are backed by the `user_moderation` table (migration 078), and admin resolutions are written to `university_audit_log` .

Real-time System

Q: How does Socket.io scale across multiple server instances?

A: Socket.io uses a Redis pub/sub adapter. We create two `IORedis` connections — `pubClient` and `subClient` — and pass them to `@socket.io/redis-adapter` . When any server instance emits `io.to('uni:abc').emit(...)` , the adapter publishes to Redis; the adapter on all other instances receives it and forwards to connected sockets in that room. This allows horizontal scaling without sticky sessions.

Q: How does the client know when to reconnect after a token expires?

A: The Socket.io middleware on the server calls `tokenService.verifyAccessTokenWithExpiry(token)` . If the token is expired (not just invalid), it sends back `TOKEN_EXPIRED` error. The client socket listens for connection errors, detects

`TOKEN_EXPIRED` , silently refreshes the access token via the HTTP interceptor, then reconnects with the new token.

Q: What are Socket.io rooms used for?

A: Three room types: `uni:{universityId}` for university-wide broadcasts (new posts, events, shuttle locations), `user:{userId}` for personal events (notification received, connection request, direct message), and `conv:{conversationId}` for chat messages. When a socket connects, it automatically joins its university room and personal room. It joins conversation rooms only when actively viewing a conversation.

Background Jobs

Q: Why use Bull queues instead of just `await` on async operations?

A: HTTP handlers must respond quickly. If sending an email takes 2 seconds, the user waits. With Bull, we enqueue the job (milliseconds), respond to the user, and the worker processes it asynchronously. Bull also provides retry logic, exponential backoff, job persistence in Redis (survives server restart), concurrency control, and job monitoring. A failed email automatically retries three times before giving up.

Q: What happens if the Bull worker crashes mid-job?

A: Bull marks the job as `active` when a worker picks it up. If the worker crashes without completing, Bull moves the job back to `waiting` after a stall timeout. On restart, the worker picks it up again. This is why our workers are idempotent — re-running them doesn't duplicate emails or notifications.

Q: How does the feed `hot_score` work?

A: We adapted the Hacker News algorithm. $hot_score = (reactions + comments \times 2) / (age_hours + 2)^{1.8}$. The denominator grows as the post ages, so older posts decay. Reaction and comment counts are maintained transactionally on every write (not computed at read time). The `feed-ranking` Bull cron runs hourly and updates `hot_score` . The "Top" feed then sorts by this pre-computed column using a GIN/btree index — no subquery aggregation at read time.

Database

Q: Why are there two migration 072 files?

A: This was a mistake made during parallel development — two features (`post_lifecycle` columns and `username` column) were given the same migration number. Knex runs migrations by filename,

so both run, and they don't conflict functionally. Going forward, we start from `077_` to avoid re-using any prefix. The lesson: coordinate migration numbering in the team's task tracker before branching.

Q: Why use generated stored columns for full-text search instead of a separate search index?

A: Generated STORED columns compute the `tsvector` value at write time (INSERT/UPDATE) and persist it in the row. The GIN index is built on this stored value. At read time, the query `WHERE search_vector @@ plainto_tsquery('english', ?)` uses the index — $O(\log n)$ not $O(n)$. Alternatives like Elasticsearch add operational complexity (separate cluster, sync pipeline). For our scale, Postgres full-text search is sufficient and has zero additional infrastructure cost.

Q: Why don't you use Postgres Row Level Security (RLS)?

A: RLS would automatically filter by university at the database driver level, which is more secure. We chose application-level filtering because: (1) RLS requires setting a session variable per request which complicates connection pooling, (2) debugging RLS policies is harder than reading service code, (3) our service layer is consistent — every query includes `where({ university_id })`. For a production system at larger scale, RLS would be the next evolution.

Frontend

Q: Why TanStack Query instead of Redux for server state?

A: Redux is general-purpose state — you have to manually handle loading, error, caching, and refetching. TanStack Query is built specifically for server state. It gives us automatic background refetch, stale-while-revalidate caching, pagination, optimistic updates, and request deduplication out of the box. We use Zustand only for truly global client state (auth token, socket connection, theme). The rule is: server data → TanStack Query, UI state → Zustand.

Q: How are routes protected? Can an unauthenticated user access the feed?

A: Route components are wrapped in `ProtectedRoute`, `AdminRoute`, or `GuestRoute` (from `apps/web/src/router/`). `ProtectedRoute` checks `authStore` for a valid access token. If none, it redirects to `/login`. `AdminRoute` additionally checks `role === 'admin' || role === 'faculty'`. `GuestRoute` redirects logged-in users away from login/register pages.

Q: Why are all pages lazy-loaded?

A: `React.lazy` + `Suspense` means each page is a separate code chunk loaded only when navigated to. The initial bundle is small — users don't download the admin panel code if they never visit it. The `page()` helper in `apps/web/src/router/index.tsx` wraps `React.lazy` + `Suspense` with a loading fallback so every page automatically has a loading state.

Q: How does the real-time notification badge work?

A: The `notificationsStore` (Zustand) holds an `unreadCount`. On login, it fetches the count from `GET /notifications/unread-count`. The socket listens on the `user:{userId}` room for `NOTIFICATION_NEW` events — each increments `unreadCount` in the store. The TopNav badge reads this store. When the notifications page is opened, it calls `POST /notifications/read-all` and resets the count.

Deployment & DevOps

Q: How is the application deployed?

A: The backend API is deployed to Azure App Service via GitHub Actions CI/CD triggered on pushes to `main`. The production start command runs `npm run start:prod` before starting the server, so migrations apply automatically on each deployment. The frontend is deployed to Vercel (configured via `vercel.json`). The database is hosted on Neon (managed Postgres with branching).

Q: How do Bull workers run in production?

A: In development, workers run as a separate process (`npm run start:dev:worker`) to avoid blocking the HTTP server. In production on Azure App Service (single process constraint), workers run in-process with the API server — the `workers/index.ts` barrel registers all workers. The queues use Redis (same instance) for durability regardless of whether workers are in-process or separate.

Q: What happens if the database migration fails mid-deploy?

A: Knex tracks completed migrations in a `knex_migrations` table. Each migration is a single transaction — if it fails, it rolls back. The server startup fails early and Azure redeploys to the previous version. We learned this the hard way: migration `070` added a `STORED GENERATED` column using `array_to_string()` which is only `STABLE`, not `IMMUTABLE`. Postgres requires `IMMUTABLE` functions in stored generated columns. The fix was to use `to_tsvector()` directly (which is immutable) instead of `array_to_string()`.

Q: How do you manage environment variables across environments?

A: `apps/api/.env` (local, not committed), `apps/web/.env` (local). Production secrets are managed via Azure App Service configuration for the API and Vercel environment variables for the web. The `.env.example` file documents every required variable. There is a strict separation: `JWT_SECRET` and `JWT_REFRESH_SECRET` are separate so that a compromised refresh token can't be used as an access token.

Feature-specific

Q: How does the content sync avoid re-importing the same article twice?

A: Each imported row carries an `external_id` (the WordPress post ID). Before inserting, the worker queries `WHERE university_id = ? AND external_id = ?`. If the row exists, it's skipped (idempotent). This means running the sync twice never duplicates content. The `content_attachments` table similarly deduplicates by `(university_id, source_url, attachable_type, attachable_id)`.

Q: What is the Skyvern fallback in content sync?

A: Some university websites don't expose a WordPress REST API — they're plain HTML pages. When the direct WordPress fetch fails, `content-sync.worker.ts` calls `skyvern.service.ts`, which uses Skyvern's browser automation API to navigate to the URL, extract the DOM, and return structured content. Skyvern runs a real Chromium browser, so it can scrape JavaScript-rendered pages. This is significantly slower (30–60s vs 1–2s for REST) so it's a fallback only.

Q: How does the lost-and-found feature work?

A: `lost_found_items` table with fields: `type` (lost/found), `title`, `description`, `category`, `location`, `image_url`, `status` (open/claimed), `reporter_id`. Users post items they lost or found. The campus module's routes are at `/api/v1/campus/lost-found`. Claimants contact the reporter via the messaging system. No automated matching — human-driven.

Q: How does the shuttle GPS work for students watching the map?

A: `ShuttlePage.tsx` subscribes to the `SHUTTLE_LOCATION` socket event on the `uni:{universityId}` room. When a driver broadcasts their GPS location, all students on the page receive it. Since GPS updates are every 5 seconds and sockets have some latency, the frontend uses `est_duration_min` and `cycle_minutes` (stored on the route in migration `073`) to linearly interpolate the bus position between updates, giving smooth animation.

Q: Can a user delete their account?

A: Not directly. Users submit an account deletion request with a reason (`POST /users/me/account-deletion-request` — migration `079`). Admins review requests in the admin panel and can approve or deny. This design (requested vs self-service deletion) is intentional for a university setting where student records may have administrative obligations. Users can deactivate their account (`POST /users/me/deactivate`, sets `deactivated_at`) which hides them from other users immediately.

Q: How do drafts work across different content types?

A: Migration `064` added `is_published` + lifecycle columns to posts, jobs, news, and events tables. `GET /me/drafts` is a UNION query across all four tables, filtered by `author_id = me AND`

`is_published = false` . The result is a unified list regardless of content type. This is the `drafts` module — a thin read-only endpoint that surfaces unsaved work across the entire platform.

Q: How do new users know what to do first (onboarding)?

A: `GET /users/me/progress` returns a set of first-run signals validated by `profileProgressSchema` in `packages/shared` — a weighted `profileScore` (0–100), plus granular booleans (`hasAvatar` , `hasBio` , `hasHeadline` , `hasAddedExperience` , `hasAddedEducation` , `hasMadePost` , `isVerified`) and `connectionCount` . The `OnboardingChecklist` component reads these and deep-links the user straight to the exact gap — for example "Add a bio" opens the edit-profile modal. The checklist disappears once `profileScore` reaches 100. The flags are computed server-side from the actual profile so they can never drift from reality.

Testing

Q: How are your integration tests structured?

A: `apps/api/src/__tests__/setup.ts` runs all pending Knex migrations against a test database (separate `TEST_DATABASE_URL`), seeds four canonical users (admin, faculty, alumni, student) with known credentials, and exports a `loginAs(email, password)` helper that returns `{ accessToken, cookie }` . Each test file uses `loginAs()` to get bearer tokens. All supertest requests must include `.set('x-university-domain', DOMAIN)` . Tests clean up in `afterEach` / `afterAll` hooks.

Q: Do you have frontend tests?

A: Yes — React Testing Library + Vitest + MSW (Mock Service Worker). MSW intercepts HTTP requests at the network level (not axios mocks) so tests are framework-agnostic and test behavior rather than implementation. Handlers live in `src/tests/msw/handlers.ts` . Test files are co-located with their components as `*.test.tsx` .

Quick Reference — Who Owns What

If the teacher asks about...	Talk to...
How the JWT / refresh token / auth flow works	Joydip Datta (Leader)
Why we chose this architecture / monorepo	Joydip Datta (Leader)
How Socket.io scales / real-time events	Joydip Datta (Leader)
Bull queues, background jobs, workers	Joydip Datta (Leader)
Admin panel, user management, audit logs	Joydip Datta (Leader)
File upload, presigned S3 URL pattern	Joydip Datta (Leader)
Rate limiting / API abuse protection	Joydip Datta (Leader)
Moderation (block / mute / report)	Joydip Datta (Leader)
Account deletion request flow	Joydip Datta (Leader)
Drafts (how posts/jobs/events share drafts)	Joydip Datta (Leader)
Database design decisions (why UUID, no RLS)	Joydip Datta (Leader)
Testing strategy, CI/CD, deployment	Joydip Datta (Leader)
Design system, tokens, dark/light theme	Joydip Datta (Leader)
How the feed works, hot_score algorithm	Md. Saem Ferdous (2)
Post scheduling / draft / lifecycle states	Md. Saem Ferdous (2)
Groups, join requests, member roles	Md. Saem Ferdous (2)
Notifications, news, badges, share	Md. Saem Ferdous (2)
Jobs, events, RSVP	Md. Mahfujur Rahman Himel Akon (3)
Connections graph, mutual connections	Md. Mahfujur Rahman Himel Akon (3)
Profile sections, experience, education	Md. Mahfujur Rahman Himel Akon (3)
Onboarding checklist, profile progress	Md. Mahfujur Rahman Himel Akon (3)
Mentorship, points economy, gift cards	Md. Mahfujur Rahman Himel Akon (3)
Chat messages, conversation types	Md. Monabbur Hosen Bhuiyan (4)
Presence system, online status	Md. Monabbur Hosen Bhuiyan (4)
Web Push notifications, VAPID	Md. Monabbur Hosen Bhuiyan (4)
Privacy settings, visibility tiers	Md. Monabbur Hosen Bhuiyan (4)
Full-text search, GIN indexes	Md. Nazmul Hasan Nasim (5)
Content sync, WordPress, Skyvern	Md. Nazmul Hasan Nasim (5)
Shuttle GPS, lost-and-found, explore	Md. Nazmul Hasan Nasim (5)
Settings, notification / privacy preferences	Md. Nazmul Hasan Nasim (5)

